МИНИСТЕРСТВО ОБРАЗОВАНИЯ РЕСПУБЛИКИ БЕЛАРУСЬ

УЧРЕЖДЕНИЕ ОБРАЗОВАНИЯ
«БРЕСТСКИЙГОСУДАРСТВЕННЫЙТЕХНИЧЕСКИЙУНИВЕРСИТЕТ» ФАКУЛЬТЕТ

ЭЛЕКТРОННО-ИНФОРМАЦИОННЫХ СИСТЕМ

Кафедра интеллектуальных информационных технологий

Отчет по лабораторной работе №2

Специальность ПО-13

Выполнил:
М.А. Романюк студент
группы ПО-13
Проверил:
А.А. Крощенко
доц. кафедры ИИТ,
05.03.2026

Брест 2026

Цель работы: закрепить навыки объектно-ориентированного программирования на языке Python

Задание 1:

Вариант 9. Множество вещественных чисел переменной мощности – Предусмотреть возможность объединения двух множеств, вывода на консоль элементов множества, а также метод, определяющий, принадлежит ли указанное значение множеству. Класс должен содержать методы, позволяющие добавлять и удалять элемент в/из множества. Конструктор должен позволять создавать объекты с начальной инициализацией. Реализацию множества осуществить на базе списка. Переопределить метод __eq__, выполняющий сравнение объектов данного типа.

Выполнение:

Код программы

```
class RealSet:
    def __init__(self, max_size=10, elements=None):
        self._max_size = max_size
        self._data = []
        if elements:
            for e in elements:
                self.add(e)

    def get_size(self):
        return len(self._data)

    def get_max_size(self):
        return self._max_size

    def set_max_size(self, value):
        if value < len(self._data):
            print("Ошибка: новый размер меньше текущего количества элементов")
        else:
            self._max_size = value

    def __str__(self):
        return "{" + ", ".join([str(round(x, 2)) for x in self._data]) + "}"

    def __eq__(self, other):
        if not isinstance(other, RealSet):
            return False
        return sorted(self._data) == sorted(other._data)
```

```
    def contains(self, x):
        return x in self._data

    def add(self, x):
        try:
            num = float(x)
            if num not in self._data:
                if len(self._data) < self._max_size:
                    self._data.append(num)
                else:
                    print(f"Ошибка: достигнут максимум ({self._max_size} элементов)")
            else:
                print(f"Элемент {num} уже существует")
        except:
            print("Ошибка: нужно вещественное число")

    def remove(self, x):
        try:
            self._data.remove(float(x))
        except:
            print(f"Элемент {x} не найден")

    def union(self, other):
        new_max = max(self._max_size, other._max_size)
        new_data = list(set(self._data + other._data))
        return RealSet(new_max, new_data[:new_max])


# пример
a = RealSet(5, [1.5, 2.3, 3.7])
b = RealSet(3, [2.3, 4.1])
print(f"A: {a}, размер: {a.get_size()}/{a.get_max_size()}")
a.add(5.5)
a.add(1.5)  # уже есть
print(f"A после добавлений: {a}")
print(f"2.3 в A? {a.contains(2.3)}")
c = a.union(b)
print(f"Объединение A и B: {c}")
a.remove(2.3)
print(f"A после удаления: {a}")
print(f"A == RealSet(5, [1.5, 3.7, 5.5])? {a == RealSet(5, [1.5, 3.7, 5.5])}")
```

Рисунки с результатами работы программы

```
A: {1.5, 2.3, 3.7}, размер: 3/5
Элемент 1.5 уже существует
A после добавлений: {1.5, 2.3, 3.7, 5.5}
2.3 в A? True
Объединение A и B: {1.5, 2.3, 3.7, 4.1, 5.5}
A после удаления: {1.5, 3.7, 5.5}
A == RealSet(5, [1.5, 3.7, 5.5])? True
```

Задание 2:

Вариант 9. Система Железнодорожная касса. Пассажир делает Заявку на станцию назначения, время и дату поездки. Система регистрирует Заявку и осуществляет поиск подходящего Поезда. Пассажир делает выбор Поезда и получает Счет на Оплату. Администратор вводит номера Поездов, промежуточные и конечные станции, цены.

Выполнение:

Код программы

```
from abc import ABC, abstractmethod
from datetime import datetime
from typing import List, Optional, Dict


class IAccounting(ABC):

    @abstractmethod
    def calculate_price(self, base_price: float) -> float:
        """Рассчитать цену с налогом."""


class Entity:

    def __init__(self, entity_id: int):
        self.id = entity_id


class Station(Entity):

    def __init__(self, entity_id: int, name: str, city: str):
        super().__init__(entity_id)
        self.name = name
        self.city = city

    def __str__(self) -> str:
        return f"{self.city} - {self.name}"
```

```python
class Carriage(Entity):

    def __init__(self, entity_id: int, carriage_type: str, seat_count: int):
        super().__init__(entity_id)
        self.carriage_type = carriage_type
        self.seats = []
        for i in range(seat_count):
            self.seats.append(Seat(i + 1))

    def free_seats(self) -> int:
        count = 0
        for seat in self.seats:
            if not seat.is_taken:
                count += 1
        return count


class Seat:

    def __init__(self, number: int):
        self.number = number
        self.is_taken = False

    def take(self) -> None:
        self.is_taken = True


class Train(Entity):

    def __init__(self, entity_id: int, train_number: str):
        super().__init__(entity_id)
        self.train_number = train_number
        self.route = []
        self.prices = {}
        self.carriages = []

    def add_carriage(self, carriage: Carriage) -> None:
        self.carriages.append(carriage)

    def free_seats(self) -> int:
        total = 0
        for carriage in self.carriages:
            total += carriage.free_seats()
        return total


class Passenger(Entity):

    def __init__(self, entity_id: int, name: str, phone: str):
```

```
        super().__init__(entity_id)
        self.name = name
        self.phone = phone
        self.requests = []

    def create_request(self, station: Station, date: datetime):
        new_request = Request(len(self.requests) + 1, self, station, date)
        self.requests.append(new_request)
        return new_request


class Request(Entity):

    def __init__(self, entity_id: int, passenger: Passenger, station: Station, date: datetime):
        super().__init__(entity_id)
        self.passenger = passenger
        self.station = station
        self.date = date
        self.status = "новая"

    def __str__(self) -> str:
        return f"заявка {self.id}: {self.passenger.name} -> {self.station.city}"


class Invoice(Entity):

    def __init__(self, entity_id: int, amount: float, passenger: Passenger):
        super().__init__(entity_id)
        self.amount = amount
        self.passenger = passenger
        self.is_paid = False

    def __str__(self) -> str:
        return f"счет {self.id} на {self.amount} руб"


class SimpleAccounting(IAccounting):

    def calculate_price(self, base_price: float) -> float:
        return base_price * 1.13


class Administrator:

    def __init__(self, system: "RailwayTicketSystem"):
        self.system = system

    def add_station(self, name: str, city: str) -> Station:
        station = Station(len(self.system.stations) + 1, name, city)
```

```
        self.system.stations.append(station)
        return station

    def add_train(self, number: str, route: List[Station], prices: Dict[str, float]) -> Train:
        train = Train(len(self.system.trains) + 1, number)
        train.route = route
        train.prices = prices
        train.add_carriage(Carriage(1, "купе", 36))
        train.add_carriage(Carriage(2, "плацкарт", 54))
        self.system.trains.append(train)
        print(f"поезд {number} добавлен")
        return train


class RailwayTicketSystem:

    def __init__(self):
        self.trains = []
        self.stations = []
        self.passengers = []
        self.requests = []
        self.invoices = []
        self.accounting = SimpleAccounting()
        self.admin = Administrator(self)

    def register_passenger(self, name: str, phone: str) -> Passenger:
        passenger = Passenger(len(self.passengers) + 1, name, phone)
        self.passengers.append(passenger)
        return passenger

    def find_trains(self, request_obj: Request) -> List[Train]:
        suitable = []
        for train in self.trains:
            if request_obj.station in train.route:
                suitable.append(train)
        return suitable

    def book_ticket(self, passenger_obj: Passenger, train_obj: Train) -> Optional[Invoice]:
        if train_obj.free_seats() <= 0:
            return None

        for carriage in train_obj.carriages:
            for seat in carriage.seats:
                if not seat.is_taken:
                    seat.take()
                    base = train_obj.prices.get(carriage.carriage_type, 1000)
                    total = self.accounting.calculate_price(base)
                    invoice = Invoice(len(self.invoices) + 1, total, passenger_obj)
```

```
                    self.invoices.append(invoice)
                    return invoice
        return None


def main():
    print("ЖЕЛЕЗНОДОРОЖНАЯ КАССА")

    system = RailwayTicketSystem()

    brest = system.admin.add_station("брестский вокзал", "Брест")
    minsk = system.admin.add_station("минск вокзал", "Минск")
    vitebsk = system.admin.add_station("вокзал", "Витебск")

    print("\nстанции добавлены:")

    train1 = system.admin.add_train("101", [brest, minsk, vitebsk], {"купе": 50, "плацкарт": 30})
    train2 = system.admin.add_train("202", [brest, minsk], {"купе": 20, "плацкарт": 12})

    ivan = system.register_passenger("иван петров", "+375292563625")
    print(f"\nпассажир: {ivan.name}")

    request_obj = ivan.create_request(brest, datetime(2025, 3, 10))
    system.requests.append(request_obj)
    print(f"создана: {request_obj}")

    trains = system.find_trains(request_obj)
    print(f"найдено поездов: {len(trains)}")
    for train in trains:
        print(f"  поезд {train.train_number}, " f"свободных мест: {train.free_seats()}")

    invoice_obj = system.book_ticket(ivan, train1)
    if invoice_obj:
        print(f"\n{invoice_obj}")
        print(f"осталось мест в поезде: {train1.free_seats()}")
    else:
        print("\nнет свободных мест")


if __name__ == "__main__":
    main()
```

Рисунки с результатами работы программы

```
поезд 101 добавлен
поезд 202 добавлен

пассажир: иван петров
создана: заявка 1: иван петров -> Бре
найдено поездов: 2
  поезд 101, свободных мест: 90
  поезд 202, свободных мест: 90

счет 1 на 56.49999999999999 руб
осталось мест в поезде: 89
```

Вывод: закрепил навыки объектно-ориентированного программирования на языке Python